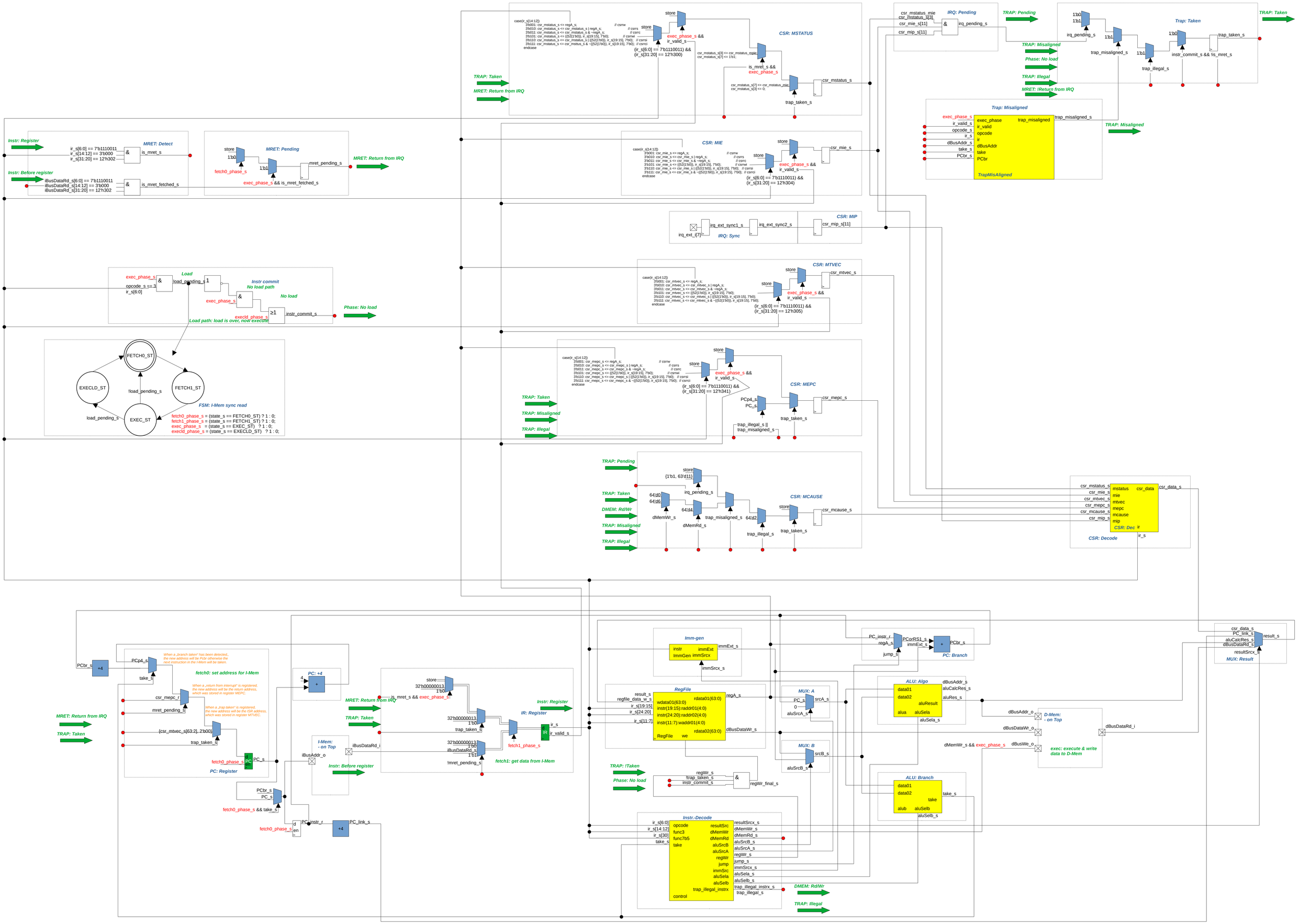
Instr: Register
ir_s[6:0] == 7'b1110011
ir_s[14:12] == 3'b000
ir_s[31:20] == 12'h302
&
is_mret_s
MRET: Detect
Instr: Before register
iBusDataRd_s[6:0] == 7'b1110011
iBusDataRd_s[14:12] == 3'b000
iBusDataRd_s[31:20] == 12'h302
&
is_mret_fetched_s
store
1'b0
fetch0_phase_s
1'b1
mret_pending_s
exec_phase_s && is_mret_fetched_s
MRET: Pending
MRET: Return from IRQ
exec_phase_s
opcode_s == 3
ir_s[6:0]
&
load_pending_s
1
Load
Instr commit
No load path
No load
exec_phase_s
&
execld_phase_s
≥1
instr_commit_s
Phase: No load
Load path: load is over, now execute
FETCH0_ST
EXECLD_ST
FETCH1_ST
!load_pending_s
load_pending_s
EXEC_ST
FSM: I-Mem sync read
fetch0_phase_s = (state_s == FETCH0_ST) ? 1 : 0;
fetch1_phase_s = (state_s == FETCH1_ST) ? 1 : 0;
exec_phase_s = (state_s == EXEC_ST) ? 1 : 0;
execld_phase_s = (state_s == EXECLD_ST) ? 1 : 0;
case(ir_s[14:12])
3'b001: csr_mstatus_s <= regA_s;
3'b010: csr_mstatus_s <= csr_mstatus_s | regA_s;
3'b011: csr_mstatus_s <= csr_mstatus_s & ~regA_s;
3'b101: csr_mstatus_s <= {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b110: csr_mstatus_s <= csr_mstatus_s | {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b111: csr_mstatus_s <= csr_mstatus_s & ~{{52{1'b0}}, ir_s[19:15], 7'b0};
endcase
// csrw
// csrrs
// csrc
// csrrw
// csrrsi
// csrrci
store
exec_phase_s && ir_valid_s
(ir_s[6:0] == 7'b1110011) && (ir_s[31:20] == 12'h300)
CSR: MSTATUS
csr_mstatus_s[3] <= csr_mstatus_mpie;
csr_mstatus_s[7] <= 1'b1;
is_mret_s && exec_phase_s
csr_mstatus_s[7] <= csr_mstatus_mpie;
csr_mstatus_s[3] <= 0;
csr_mstatus_s
trap_taken_s
TRAP: Taken
MRET: Return from IRQ
csr_mstatus_mie
csr_mstatus_s[3]
csr_mie_s[11]
csr_mip_s[11]
&
IRQ: Pending
irq_pending_s
TRAP: Pending
TRAP: Misaligned
Phase: No load
TRAP: Illegal
MRET: !Return from IRQ
1'b0
1'b1
irq_pending_s
1'b1
trap_misaligned_s
1'b1
trap_illegal_s
1'b0
trap_taken_s
instr_commit_s && !is_mret_s
Trap: Taken
TRAP: Taken
Trap: Misaligned
exec_phase_s
ir_valid_s
opcode_s
ir_s
dBusAddr_s
take_s
PCbr_s
exec_phase
ir_valid
opcode
ir
dBusAddr
take
PCbr
trap_misaligned
TrapMisAligned
trap_misaligned_s
TRAP: Misaligned
CSR: MIE
case(ir_s[14:12])
3'b001: csr_mie_s <= regA_s;
3'b010: csr_mie_s <= csr_mie_s | regA_s;
3'b011: csr_mie_s <= csr_mie_s & ~regA_s;
3'b101: csr_mie_s <= {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b110: csr_mie_s <= csr_mie_s | {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b111: csr_mie_s <= csr_mie_s & ~{{52{1'b0}}, ir_s[19:15], 7'b0};
endcase
// csrw
// csrrs
// csrc
// csrrw
// csrrsi
// csrrci
store
exec_phase_s && ir_valid_s
(ir_s[6:0] == 7'b1110011) && (ir_s[31:20] == 12'h304)
csr_mie_s
CSR: MIP
irq_ext_i[7]
irq_ext_sync1_s
irq_ext_sync2_s
csr_mip_s[11]
IRQ: Sync
CSR: MTVEC
case(ir_s[14:12])
3'b001: csr_mtvec_s <= regA_s;
3'b010: csr_mtvec_s <= csr_mtvec_s | regA_s;
3'b011: csr_mtvec_s <= csr_mtvec_s & ~regA_s;
3'b101: csr_mtvec_s <= {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b110: csr_mtvec_s <= csr_mtvec_s | {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b111: csr_mtvec_s <= csr_mtvec_s & ~{{52{1'b0}}, ir_s[19:15], 7'b0};
endcase
store
exec_phase_s && ir_valid_s
(ir_s[6:0] == 7'b1110011) && (ir_s[31:20] == 12'h305)
csr_mtvec_s
case(ir_s[14:12])
3'b001: csr_mepc_s <= regA_s;
3'b010: csr_mepc_s <= csr_mepc_s | regA_s;
3'b011: csr_mepc_s <= csr_mepc_s & ~regA_s;
3'b101: csr_mepc_s <= {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b110: csr_mepc_s <= csr_mepc_s | {{52{1'b0}}, ir_s[19:15], 7'b0};
3'b111: csr_mepc_s <= csr_mepc_s & ~{{52{1'b0}}, ir_s[19:15], 7'b0};
endcase
// csrw
// csrrs
// csrc
// csrrw
// csrrsi
// csrrci
store
exec_phase_s && ir_valid_s
(ir_s[6:0] == 7'b1110011) && (ir_s[31:20] == 12'h341)
CSR: MEPC
PCp4_s
PC_s
csr_mepc_s
trap_taken_s
trap_illegal_s || trap_misaligned_s
TRAP: Taken
TRAP: Misaligned
TRAP: Illegal
TRAP: Pending
DMEM: Rd/Wr
store
{1'b1, 63'd11}
irq_pending_s
64'd0
64'd6
64'd4
64'd2
dMemWr_s
dMemRd_s
trap_misaligned_s
trap_illegal_s
trap_taken_s
CSR: MCAUSE
csr_mcause_s
csr_mstatus_s
csr_mie_s
csr_mtvec_s
csr_mepc_s
csr_mcause_s
csr_mip_s
mstatus
mie
mtvec
mepc
mcause
mip
csr_data
CSR: Dec
ir
csr_data_s
CSR: Decode
ir_s
PCbr_s
+4
PCp4_s
take_s
When a „branch taken" has been detected,, the new address will be PCbr otherwise the next instruction in the I-Mem will be taken.
fetch0: set address for I-Mem
When a „return from interupt" is registered, the new address will be the return address, which was stored in register MEPC.
When a „trap taken" is registered, the new address will be the ISR address, which was stored in register MTVEC.
csr_mepc_r
mret_pending_s
{csr_mtvec_s[63:2], 2'b00}
trap_taken_s
fetch0_phase_s
d
PC
en
PC_s
PC: Register
MRET: Return from IRQ
TRAP: Taken
PC: +4
4
+
store
32'h00000013
1'b0
is_mret_s && exec_phase_s
32'h00000013
1'b0
trap_taken_s
32'h00000013
1'b0
iBusDataRd_s
1'b1
!mret_pending_s
fetch1_phase_s
fetch1: get data from I-Mem
Instr: Register
IR: Register
d
IR
en
ir_s
ir_valid_s
I-Mem: - on Top
iBusAddr_o
iBusDataRd_i
Instr: Before register
PCbr_s
PC_s
fetch0_phase_s && take_s
fetch0_phase_s
PC_instr_r
d
en
PC_link_s
+4
Imm-gen
instr
ImmGen
immExt
immSrcx
immExt_s
immSrcx_s
RegFile
result_s
regfile_data_w_s
ir_s[19:15]
ir_s[24:20]
ir_s[11:7]
wdata01(63:0)
instr(19:15):raddr01(4:0)
instr(24:20):raddr02(4:0)
instr(11:7):waddr01(4:0)
rdata01(63:0)
rdata02(63:0)
RegFile
we
regA_s
dBusDataWr_s
MUX: A
PC_s
0
srcA_s
aluSrcA_s
MUX: B
srcB_s
aluSrcB_s
TRAP: !Taken
Phase: No load
regWr_s
!trap_taken_s
instr_commit_s
&
regWr_final_s
Instr.-Decode
ir_s[6:0]
ir_s[14:12]
ir_s[30]
take_s
opcode
func3
func7b5
take
resultSrc
dMemWr
dMemRd
aluSrcB
aluSrcA
regWr
jump
immSrc
aluSela
aluSelb
trap_illegal_instrx
control
resultSrcx_s
dMemWr_s
dMemRd_s
aluSrcB_s
aluSrcA_s
regWr_s
jump_s
immSrcx_s
aluSela_s
aluSelb_s
trap_illegal_instrx_s
trap_illegal_s
DMEM: Rd/Wr
TRAP: Illegal
PC_instr_r
regA_s
jump_s
PCorRS1_s
immExt_s
+
PCbr_s
PC: Branch
ALU: Algo
data01
data02
aluResult
alua
aluSela
dBusAddr_s
aluCalcRes_s
aluRes_s
aluSela_s
ALU: Branch
data01
data02
take
alub
aluSelb
take_s
aluSelb_s
dBusAddr_o
dBusDataWr_o
dMemWr_s && exec_phase_s
dBusWe_o
D-Mem: - on Top
dBusDataRd_i
exec: execute & write data to D-Mem
csr_data_s
PC_link_s
aluCalcRes_s
dBusDataRd_s
resultSrcx_s
result_s
MUX: Result